

WEEK 10 · JSON, LOCALSTORAGE, PERSISTENCE

IT IS STILL THERE TOMORROW

Last week the application got a shape.

Today the data stops living in the tab.

WHERE WE ARE

An application that is correct, architecturally clean — and that forgets everything when you close it.

AFTER TODAY YOU CAN

Write a storage layer that survives a reload, a corrupt key, and a change to your own data shape.

WHAT THIS UNLOCKS

Your project stops being a demo. Next week the same layer reads from a server.

The browser will keep exactly one thing for you: a string.

EVERYTHING YOU BUILT LAST WEEK IS GONE

WHAT WE HAVE

One state object, one render function, handlers that only change state. Correct, and every bug from week 8 is impossible.

WHAT A USER DOES

Adds nine items. Closes the tab.
Comes back tomorrow.

WHAT THEY FIND

Six demo items they have never seen. **There is no error and no warning** — the application is working exactly as written.

Modern web apps are data-driven: data comes from a server, data is stored temporarily or permanently in the browser, and data is exchanged in a standard format.

THE SAME APPLICATION TWICE. ONE OF THEM WRITES IT DOWN.

left: in memory · right: the same file, six lines longer↻ reload🔗 open ↗

THE SAME APPLICATION TWICE. ONE OF THEM WRITES THE COLLECTION DOWN.

add an item to bothremove the first from bothreload bothwipe the saved copy

IN MEMORY — WEEK 9'S APPLICATION

0 items · in memory only

WRITTEN DOWN — THE SAME APPLICATION, SIX LINES LONGER

0 items · saved under "demo01.pantry.v1"

Add a few items, then press "reload both".

The two panels run the same file. The lines that mention localStorage or JSON in it: 3. That is the whole of today's difference, and by the end of the lecture it lives in a file of its own.

► **ADD THREE ITEMS TO BOTH, AND ASK WHICH ONE IS BETTER WRITTEN.** Nobody can tell. Then press "reload both".

WHAT THE BROWSER IS ACTUALLY WILLING TO KEEP

```
1 localStorage.setItem('key', 'a string'); // ← the only thing it takes
2
3 localStorage.getItem('key');             // → 'a string', or null
4 localStorage.getItem('never-written');   // → null
```

STRINGS IN

Not objects, not arrays, not numbers. Anything else is coerced.

STRINGS OUT

Or `null`, which is an answer and not a failure.

SO TODAY IS TWO JOBS

Turn an object into a string safely.
Turn a string that might be **anything** back into an object safely.

JSON IS A TEXT FORMAT, AND IT HAS FIVE RULES

JAVASCRIPT

```
{  
  id: 'a1',  
  title: 'קמח מלא',  
  number: 2,  
  fresh: true,  
}
```

JSON

```
{  
  "id": "a1",  
  "title": "קמח מלא",  
  "number": 2,  
  "fresh": true  
}
```

1

Keys must be **strings**

2

Double quotes only

3

No functions

4

No comments

5

No trailing commas

THE ROUND TRIP, FIELD BY FIELD

the object · the string · what came back, compared

↻ reload

open ↗

AN OBJECT OUT THROUGH STRINGIFY, BACK THROUGH PARSE, AND COMPARED FIELD BY FIELD

THE OBJECT (EDITABLE)

JavaScript, evaluated on every keystroke

```
((
  id: 'a1',
  title: 'קמח חלא',
  number: 2,
  fresh: true,
  note: null,
  tags: ['בשים', 'בשים'],
  shelf: { room: 'חטבח', height: 3 },
  created: new Date('2026-09-01T08:00:00Z'),
```

THE STRING LOCALSTORAGE WOULD HOLD

```
{
  "id": "a1",
  "title": "קמח חלא",
  "number": 2,
  "fresh": true,
  "note": null,
  "tags": [
    "בשים",
    "בשים"
  ]
}
```

276 bytes — this string is all localStorage ever holds

FIELD	BEFORE — VALUE AND TYPEOF	AFTER — VALUE AND TYPEOF	SURVIVED?
id	"a1" · string	"a1" · string	yes
title	"קמח חלא" · string	"קמח חלא" · string	yes
number	2 · number	2 · number	yes
fresh	true · boolean	true · boolean	yes
note	null · null	null · null	yes
tags	["בשים","בשים"] · object	["בשים","בשים"] · object	yes
shelf	{ "room": "חטבח", "height": 3 } · object	{ "room": "חטבח", "height": 3 } · object	yes
created	Date(2026-09-01T08:00:00.000Z) · object	"2026-09-01T08:00:00.000Z" · string	NO
restock	undefined · undefined	- gone -	NO

► Read the created row on its own. **WHAT TYPE WENT IN? WHAT CAME BACK?**

TWELVE VALUES, THREE DIFFERENT ANSWERS EACH

alone · as a property · as an array member — real return values

↻ reload

open ↗

TWELVE VALUES, THREE WAYS. EVERY CELL IS A REAL RETURN VALUE.

THE VALUE	STRINGIFY(V)	STRINGIFY({ V })	STRINGIFY([V])	WHAT HAPPENED
a string	"mnp"	{"v":"mnp"}	["mnp"]	kept
a number	2	{"v":2}	[2]	kept
true	true	{"v":true}	[true]	kept
null	null	{"v":null}	[null]	kept
undefined	undefined (not a string!)	{}	[null]	DROPPED as a property, null in an array
a function	undefined (not a string!)	{}	[null]	DROPPED as a property, null in an array
a Symbol	undefined (not a string!)	{}	[null]	DROPPED as a property, null in an array
NaN	null	{"v":null}	[null]	becomes null — the number is gone, quietly
Infinity	null	{"v":null}	[null]	becomes null — same
a Date	"2026-09-01T08:00:00.000Z"	{"v":"2026-09-01T08:00:00.000Z"}	["2026-09-01T08:00:00.000Z"]	becomes a STRING, via Date.toJSON
a Map	{}	{"v":{}}	[{}]	becomes {} — the entries are gone
a Set	{}	{"v":{}}	[{}]	becomes {} — the members are gone
a BigInt	throws TypeError	throws TypeError	throws TypeError	THROWS — TypeError
a circular object	throws TypeError	throws TypeError	throws TypeError	THROWS — TypeError

8 of these 14 values change meaning WITHOUT SAYING SO. 2 of them throw, and those are the ones you will fix today rather than in March.

The two rows that **throw** are the lucky ones. Everything else on this table fails silently, which means it fails later, somewhere else, in a function that has nothing to do with storage.

► **COVER THE LAST COLUMN AND TAKE A VOTE ON THE FUNCTION ROW.** Then look at the two rows that **THROW** — those are the lucky ones.

THE DATE, AND THE CRASH THAT ARRIVES TOMORROW

WHAT EVERYBODY WRITES

```
1 const item = { created: new Date() };
2
3 save(item);
4 // ... tomorrow, after a reload ...
5 item.created.getFullYear();
6 // TypeError: created.getFullYear
7 //           is not a function
```

Works perfectly. Until the first reload.

THE RULE

```
1 const item = { created: new Date().toISOString() };
2
3 // and where you display it, and nowhere else:
4 new Date(item.created).toLocaleDateString('he-IL');
```

Store the string. Convert at the point of use.

THE SECOND ARGUMENT, AND WHAT IT COSTS

replacer as an array · as a function · and the reviver

reload open

STRINGIFY(VALUE, REPLACER) AND PARSE(TEXT, REVIVER)

no replacer replacer as an array — an allowlist replacer as a function reviver — bring the Date back

THE CALL
JSON.stringify(item, null, 2)
JSON.parse(text)

WHAT WAS WRITTEN

```
{
  "id": "a1",
  "title": "אלקח חלק",
  "number": 2,
  "secret": "do-not-save-me",
  "created": "2026-09-01T08:00:00.000Z",
  "shelf": {
    "room": "מטבח",
    "height": 3
  }
}
```

188 bytes

WHAT CAME BACK — VALUE AND TYPEOF
id: "a1" · string
title: "אלקח חלק" · string
number: 2 · number
secret: "do-not-save-me" · string
created: "2026-09-01T08:00:00.000Z" · string
shelf: room: "מטבח" · string
height: 3 · number

The Date is a string on the way back, and `secret` went to disk with everything else.

► The reviver gets the Date back — by **GUESSING FROM THE SHAPE OF A STRING**. Ask what happens the day somebody stores a note that looks like a timestamp.

READING A NESTED STRUCTURE YOU DID NOT WRITE

OPTIMISTIC

```
const city = data.user.address.city;  
const first = data.items[0].title;
```

Two `TypeError` crashes waiting for a payload that is one field short.

DEFENSIVE AT THE EDGE

```
const city = data?.user?.address?.city ?? 'unknown';  
const first = data?.items?.[0]?.title ?? null;
```

Good for reading. Not a substitute for checking the shape — act 5.

? . stops a crash. It does not tell you the data is right.



THE WHOLE API, AND THERE IS NO SEVENTH LINE

```
1 localStorage.setItem(key, value); // store a key/value pair
2 localStorage.getItem(key);        // get the value by key
3 localStorage.removeItem(key);      // remove the key with its value
4 localStorage.clear();              // delete EVERYTHING for this origin
5 localStorage.key(index);           // the key at a given position
6 localStorage.length;               // how many are stored
```

► `clear()` is the one to be frightened of. It empties the **ORIGIN**, not your page.

TWO MISTAKES, BOTH SILENT, BOTH IN THE FIRST HOUR

left: setItem with an object · right: the very first read

↺ reload

open ↗

TWO MISTAKES, BOTH SILENT, BOTH IN THE FIRST HOUR

WRITING — SETITEM TAKES A STRING

store the object

store JSON.stringify(object)

WHAT WAS STORED

WHAT IS ON DISK

BYTES

READING — THE FIRST VISIT

the unguarded read

the honest read

```
const raw = localStorage.getItem('demo05.never-written')
raw        → null   (typeof object)
JSON.parse(raw) → null ← IT DID NOT THROW

let items = JSON.parse(raw)
items.map(row) → ?
               → TypeError: Cannot read properties of null (reading 'map')
```

The parse was fine. The crash is four lines later, in render, and it says nothing about storage — which is why the next half hour goes into render.

Neither of these is an error the browser will tell you about. The first produces a string that is the same for every object in the world; the second produces null, which is a perfectly good value right up to the line that treats it as an array.

► **STORE THREE DIFFERENT OBJECTS.** Three went in. How many distinct strings came out?

THREE PLACES TO PUT A STRING. ONE SLIDE.

	LOCALSTORAGE	SESSIONSTORAGE	A COOKIE
survives a reload	yes	yes	until <code>max-age</code>
survives closing the browser	yes	no	until <code>max-age</code>
shared with another tab	yes	no — per tab	yes
sent to the server	no	no	on every single request
room	~5MB	~5MB	~4KB, refused in silence

The row that decides it is “sent to the server”.

ONE ORIGIN, AND EVERYTHING YOU HAVE EVER BUILT IS IN IT

every key this origin already holds • and 5KB into a cookie

reload

open

THE SAME STRING IN THREE PLACES, ON ONE ORIGIN

origin: http://127.0.0.1:52085 - path: /weeks/week-10-json-storage/demos/07-where-it-lives/index.html — the path is NOT part of the scope.

write "shalom" into all three

write 5KB into all three

read all three

remove what this demo wrote

	LOCALSTORAGE	SESSIONSTORAGE	COOKIE
what came back	null	null	null
size stored	—	—	—
survives a reload	yes	yes	until max-age
survives closing the browser	yes	NO	until max-age
shared with another tab	yes	NO — per tab	yes
sent to the server	no	no	no
room	~5MB per origin	~5MB per origin	~4KB, and refused silently

EVERY KEY THIS ORIGIN IS ALREADY HOLDING

demo09.debounce
demo09.every
demo09.guard

localStorage is scoped to the ORIGIN — scheme, host and port — and not to the folder. Every project you have ever opened with Live Server on this machine is in that list. That is the whole argument for a namespaced key.

sessionStorage is per TAB, and no page can show you that from inside itself. Open this file in a second tab, press "read" in both, and compare: localStorage agrees across the two, sessionStorage does not. Duplicating a tab copies it once and then the two diverge.

▶ localStorage is scoped to **SCHEME + HOST + PORT**, not to the folder. Every Live Server project on this machine shares one drawer.

HOW MUCH ROOM IS THERE, ACTUALLY

write 64KB blocks until it says no, and read the error

↺ reload

open ↗

FILL IT UNTIL IT SAYS NO

fill localStorage

clean up

this origin is currently holding 0.6 KB

Nothing written yet.

SAVE() WITH AN EMPTY CATCH

```
function save(items) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(items));  
  } catch {}  
}
```

the application says nothing

SAVE() THAT RETURNS WHAT HAPPENED

```
function save(state) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(payload));  
    return 'saved';  
  } catch (error) {  
    console.warn('[storage] write failed', error);  
    return 'full';  
  }  
}
```

the application says nothing yet either

The quota belongs to the **origin**, not to this page — so what was measured is what was left over after every other project you have run on this host and port. Two laptops in the same room give two different numbers, and both are right.

► The name is `QuotaExceededError` here and `NS_ERROR_DOM_QUOTA_REACHED` in Firefox. **BRANCH ON “THE WRITE FAILED”, NEVER ON THE NAME.**

WHAT LOCALSTORAGE IS NOT

NOT A DATABASE

No queries, no indexes, no transactions. You read the **whole thing** and you write the **whole thing**.

NOT ASYNCHRONOUS

Every read and every write blocks the main thread. That is fine for a few hundred items and it is act 6's subject.

NOT SECURE

Every script on the page can read all of it. **No passwords, no tokens**, no anything you would mind a stranger seeing.

It is a drawer, and the whole drawer comes out every time.

LIVE CODING · 1 OF 2

BUILD THE LAYER, AND WATCH THE DRAWER

- 1 New file `js/storage.js`. A key with a namespace and a version.
- 2 `save(state)` — stringify **the collection only**.
- 3 `load()` — and the first version, deliberately unguarded.
- 4 `app.js`: **one line**. `subscribe(persist)`, beside the `subscribe(render)` that was already there.
- 5 **DevTools > Application > Local Storage.**
Add an item, and watch the key appear. Add another, and watch the value change.

► Then reload. **THEN DELETE ONE CHARACTER FROM THE VALUE, BY HAND, AND RELOAD AGAIN.**

THE ACCESS THROWS, NOT JUST THE PARSE

WHAT EVERYBODY WRITES

```
const raw = localStorage.getItem(KEY);  
try {  
  return JSON.parse(raw);  
} catch {  
  return null;  
}
```

Line 1 is outside the `try`, and line 1 is the one that throws in a browser with storage blocked.

THE LAYER

```
let raw;  
try {  
  raw = localStorage.getItem(KEY);  
} catch {  
  return null; // blocked. Draw the page anyway.  
}
```

Private mode, blocked cookies, partitioned storage — all the same throw.

A throw on the boot path is a blank page.

ABSENCE IS NOT CORRUPTION

WHAT HAPPENED	WHAT <code>LOAD()</code> RETURNS	AND WHY
<code>getItem</code> threw	<code>null</code>	storage is blocked. Not a reason to fail to draw
<code>getItem</code> returned <code>null</code>	<code>null</code>	a first visit. The normal answer
<code>JSON.parse</code> threw	<code>null</code>	corrupt — and leave the string alone
the shape is not ours	<code>null</code>	valid JSON is not valid data
the saved collection is empty	<code>[]</code>	an answer: the user deleted everything

AN EMPTY CATCH IS WORSE THAN NO CATCH

SILENT

```
function save(items) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(items));  
  } catch {}  
}
```

IT SAYS WHAT HAPPENED

```
function save(state) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(payload));  
    return 'saved';  
  } catch (error) {  
    console.warn('[storage] write failed', error);  
    return 'full';  
  }  
}
```

The user has an application that **looks exactly like one that is saving**. They find out when they close the tab.

A return value the UI can draw a sentence from.

SIX THINGS THAT ARE NOT YOUR DATA

the same six payloads through two loaders

↺ reload

open ↗

THE SAME SIX PAYLOADS THROUGH TWO LOADERS

1 · truncated

2 · nothing saved

3 · the string null

4 · valid JSON, wrong shape

5 · one bad member

6 · a version from the future

0 · perfectly fine

WHAT IS ON DISK

```
{"v":1,"items":[{"id":"a1","tit
```

THE LOADER EVERYBODY WRITES

```
function load() {  
  const raw = localStorage.getItem(KEY);  
  return JSON.parse(raw).items;  
}
```

THREW **SyntaxError: Unterminated string in JSON at position 31 (line 1 column 32)**

THE LOADER IN THE ASSIGNMENT

```
function load() {  
  let raw;  
  try { raw = localStorage.getItem(KEY); } catch { return null; }  
  if (raw === null) return null; // 2  
  let parsed;  
  try { parsed = JSON.parse(raw); } catch { return null; } // 1  
  return migrate(parsed); // 3, 4, 5, 6  
}
```

returned null — the caller seeds instead, and the page draws

נתוני דוגמה

1

payloads run: 1 - the naive loader produced a usable collection 0 of those times.

► **RUN 1 AND 2 AND LET THE ROOM PROPOSE THE FIXES.** Write both on the board. Then run 3.

PARSE, THEN CHECK — AND CHECK ONLY WHAT YOU READ

```
1  const isItem = (value) =>
2    value !== null &&
3    typeof value === 'object' &&
4    typeof value.id === 'string' && value.id !== '' &&
5    typeof value.title === 'string' &&
6    Number.isFinite(value.number) &&
7    typeof value.category === 'string';
8
9  return parsed.items.filter(isItem); // keep the good ones, drop the rest
```

► **THE FOUR FIELDS THE APPLICATION ACTUALLY READS, WITH THE TYPES IT ACTUALLY ASSUMES. NOTHING MORE.** A validator that mirrors the whole schema is a second definition of the item.

“CLEAR ALL” IS NOT `removeItem`

THE BEST BUG OF THE WEEK

```
clear.addEventListener('click', () => {  
  if (!confirm('Delete all?')) return;  
  localStorage.removeItem(KEY);  
  setState({ items: [] });  
});
```

Forty real items deleted. **Reload, and six demo items are back** — because a missing key means “this browser has never been here”.

CLEARING IS SAVING

```
clear.addEventListener('click', () => {  
  if (!confirm('Delete all?')) return;  
  setState({ items: [] });  
  // `persist` writes the empty collection  
});
```

`removeItem` belongs to one control only: **“pretend I have never been here”**.

WHERE DOES `save()` GO?

WHAT EVERYBODY WRITES FIRST

```
function onAdd() { ...; save(); }  
function onEdit() { ...; save(); }  
function onRemove() { ...; save(); }  
function onBump() { ...; save(); }
```

It works — and a **fifth** handler is a fifth chance to forget, invisible until a reload.

ONE LINE, ONCE

```
subscribe(render);  
subscribe(persist);
```

Not one handler learns that persistence exists. **They cannot forget, because they were never told.**

The same argument as week 9's handlers that draw, one layer further out.

THE WHOLE OF TODAY, AND IT IS FIVE LINES

```
1 hydrate(load() ?? SEED_ITEMS); // ← the only line that changed
2
3 subscribe(render);
4 subscribe(persist);           // ← the only line that was added
5 wire();
6 render(getState());
```

render.js

Not opened.

events.js

Not opened.

state.js

Two new fields, neither about saving.

WHAT A KEYSTROKE COSTS

one word typed into a search box, three saving strategies ↻ reload open ↗

ONE WORD TYPED INTO A SEARCH BOX, THREE SAVING STRATEGIES

search type "פפריקה" for me add an item (a real change) close the tab NOW reset

STRATEGY	WRITES	TOTAL MS BLOCKING THE MAIN THREAD	ON DISK RIGHT NOW	CORRECT?
every change	0	0.0	2 items	yes
identity guard	0	0.0	2 items	yes
debounce 300ms	0	0.0	2 items	yes

Type something, or press the button, and watch the first two columns separate.

localStorage is **synchronous**. The millisecond column is time the page spent unable to do anything else — not time in the background.

▶ `subscribe(persist)` is correct — and after week 9, **TYPING IS A STATE CHANGE**. Press the button and watch the first column climb.

TWO FIXES, AND BOTH ARE RIGHT

THE IDENTITY GUARD

```
let lastSaved = null;

export function save(state) {
  if (state.items === lastSaved) return 'unchanged';
  ...
  lastSaved = state.items;
}
```

Exact, and free. It works because week 9's handlers build new arrays instead of editing the one in state.

THE DEBOUNCE

```
let timer;
const saveSoon = (state) => {
  clearTimeout(timer);
  timer = setTimeout(() => save(state), 300);
};

addEventListener('pagehide', flush); // ← not optional
```

The general tool. **pagehide** closes the window in which the change is on screen and not on disk.

WHAT BELONGS IN STATE, AND WHAT BELONGS ON DISK

STATE — SIX FIELDS

- 1 items — **the collection**
- 2 query, category, sort
- 3 editingId — which row is open
- 4 storageError — did the write fail

DISK — ONE

items.

Restore a filter the user forgot they set and they see one row and no explanation — **a bug that reads exactly like “my data disappeared”**.

Those are two different questions.

THE LETTER OF CRUD THAT WAS MISSING

C, R, D – WEEK 8 AND 9

The form adds, `render` reads, the row button removes.

U – TODAY

One form, two jobs. `editingId` in state decides which; the submit handler is one `if`.

AND THE RULE IT FORCES

Data is drawn from state. A control's own value is not.

```
1  setState({  
2    items: state.items.map((item) => (item.id === editing.id ? { ...item, ...fields } : item)),  
3    editingId: null,  
4  });
```

LIVE CODING · 2 OF 2

GUARD IT, AND FINISH THE CYCLE

- 1 Go back to the unguarded `load()` from break 1 and **fix it in front of them**: the access, the `null`, the parse, the shape.
- 2 Corrupt the key by hand in **Application** after each step, and reload. Four payloads, four different survivals.
- 3 The edit button, and the submit handler's one `if`.
- 4 “Clear all” — and **reload straight afterwards**.

► Keep **APPLICATION** open the whole time. Every change on screen should have a visible twin in the drawer.

THE SHAPE CHANGED. THEIR BROWSERS DID NOT.

MONDAY

You ship. People use it. Their data is written in **version 1**.

FRIDAY

An item gains a field. Your code reads it everywhere.

FRIDAY EVENING

Every one of them still holds version 1 — and it is **the only copy of their data there is**.

**A version number costs one field,
and it is written before you need it.**

YESTERDAY'S DATA, THREE PROGRAMS

crashes · silently wrong · migrated

reload open

THE SHAPE CHANGED. THEIR BROWSERS DID NOT.

put v0 on disk — a bare array put v1 on disk — no tags field put v2 on disk — the current shape

WHAT IS ON DISK

```
{ "v": 1, "items": [ { "id": "a1", "title": "קמח מלא", "number": 2, "category": "יבשים" }, { "id": "a2", "title": "פפריקה מעושנת", "number": 3, "category": "תבלינים" } ] }
```

V2 CODE, NO MIGRATION

```
items.map((i) => i.tags.length)
```

THREW TypeError: Cannot read properties of undefined (reading 'length')

V2 CODE, GUARDED WITH ?.

```
items.map((i) => i.tags?.length ?? 0)
```

קמח מלא	0 tags
פפריקה מעושנת	0 tags

No error, no blank page, and 2 of 2 rows are **WRONG**. Nothing on this screen says so, and nothing ever will.

V2 CODE, WITH A MIGRATION

```
if (data.v === 1) {  
  items = data.items.map(  
    (i) => ({ ...i, tags: [i.category] }));  
  save({ v: 2, items }); // once  
}
```

קמח מלא	1 tags
פפריקה מעושנת	1 tags

v1 → v2

on disk now: { "v": 2, "items": [{ "id": "a1", "title": "קמח מלא", "number": 2, "cat... — the migration ran ONCE. Reload and the v1 branch is not taken.

A version number costs one field and is written before you need it. The alternative is finding out from a user which of three shapes their browser is holding.

► **STAY ON THE MIDDLE PANEL FOR A MINUTE.** No error, no blank page, and every row is wrong.

THREE ANSWERS, NOT TWO

```
1 function migrate(parsed) {  
2   if (Array.isArray(parsed)) return parsed.filter(isItem);           // v0: adopt it  
3  
4   if (parsed?.v === VERSION) return parsed.items.filter(isItem);    // current  
5  
6   // A version we have never heard of. Data written by a NEWER version may mean  
7   // something different in every field – a guess CORRUPTS rather than loses.  
8   return null;  
9 }
```

► The bare array is **NOT HYPOTHETICAL**. It is what your own first attempt writes, this afternoon, before part T.

TWO FAULTS IN A STORAGE LAYER THE MACHINE WROTE

```
1  const KEY = 'items';
2
3  export function load() {
4    try {
5      return JSON.parse(localStorage.getItem(KEY)) || [];
6    } catch (e) {
7      localStorage.removeItem(KEY);
8      return [];
9    }
10 }
11
12 export function save(items) {
13   try { localStorage.setItem(KEY, JSON.stringify(items)); } catch {}
14 }
```

► It runs. It passes every manual test. **FIND THE TWO THAT WILL COST A USER THEIR DATA** — and the third that will cost you an afternoon.

THREE QUESTIONS BEFORE YOU GO

- 1 A user deletes every item and reloads.
What has to be true of `load()` for the demo data not to come back?
- 2 `JSON.parse` succeeded.
What do you now know about the data, and what do you not?
- 3 You add a fifth handler to `events.js` next month.
How many lines do you have to write for it to be saved?



TODAY'S HANDS-ON, AND WHAT NEXT WEEK OPENS

10 MINUTES · WARM-UP

Five small functions, four of them wrong. All five are pieces of the file you are about to write. No git, no marks.

80 MINUTES · YOUR PROJECT

`js/storage.js`, the edit cycle, the corruption guard and “clear all”. **Keep Application open the whole time.**

BEFORE YOU LEAVE

Corrupt your own key by hand and reload. If the page comes up, you are finished. **If it does not, you are not.**

► Next week the data stops coming from disk and starts coming from **SOMEBODY ELSE'S COMPUTER** — which is slow, which fails, and which is the same layer one more time.